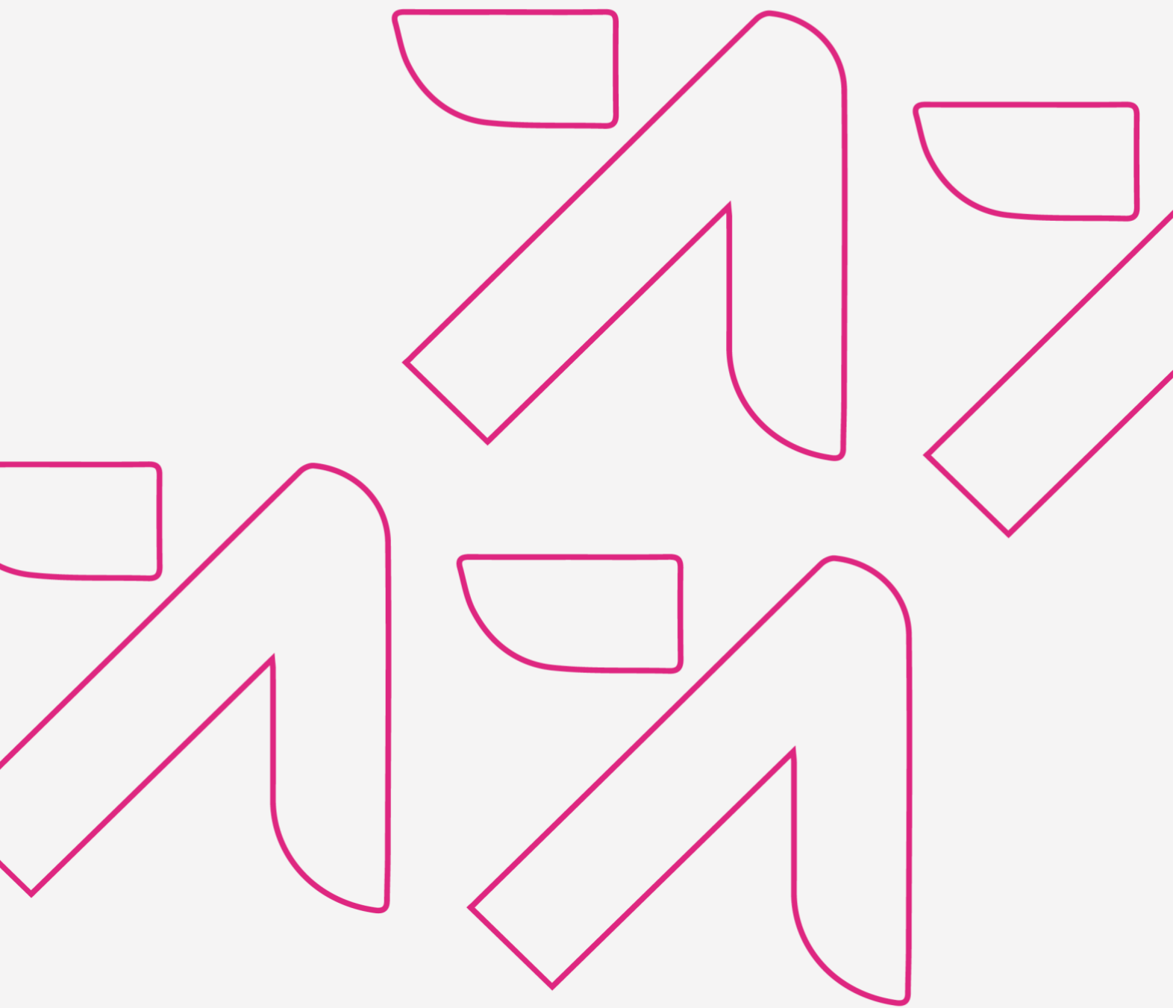




Máster de

# Ciberseguridad Hacking Ético

[Módulo Bastionado de Redes y Sistemas]

Curso académico 2026/2027 - Edición Presencial

## UT9. Gestión de parches y vulnerabilidades

**Encaje en el curso 2026/2027:** esta unidad desarrolla la parte de actualizaciones del RA7 (tema 13) y es la continuación natural del análisis de riesgos del RA1.

### 1. Objetivo de la gestión de parches

El objetivo del parcheo es mantener sistemas y aplicaciones protegidos frente a vulnerabilidades conocidas, evitando que se conviertan en vectores de ataque explotables. Un sistema sin parches es un riesgo directo y predecible: los atacantes suelen explotar CVEs públicas que llevan meses (o años) sin corregirse.

El proceso de actualización debe ser continuo, planificado y medible.

### 2. Concepto de vulnerabilidad y CVE

#### 2.1. Vulnerabilidad

Debilidad técnica en software, hardware, configuración o diseño que permite a un atacante comprometer confidencialidad, integridad o disponibilidad.

#### 2.2. CVE (Common Vulnerabilities and Exposures)

Es un identificador único que referencia una vulnerabilidad pública. Ejemplo: **CVE-2024-3094**, la puerta trasera introducida en la librería XZ Utils (CVSS 10.0).

Proporciona:

- Identificador
- Descripción
- Productos afectados
- Enlaces a parches y avisos

#### 2.3. CVSS (Common Vulnerability Scoring System)

Un sistema de puntuación para medir criticidad:

- 0.1–3.9 → Bajo
- 4.0–6.9 → Medio
- 7.0–8.9 → Alto
- 9.0–10.0 → Crítico

Conviven dos versiones: CVSS 3.1, todavía la más extendida, y **CVSS 4.0**, que afina mejor el contexto. Fíjate siempre en cuál se está usando.

#### 2.4. Más allá de CVSS: probabilidad real de explotación

CVSS mide gravedad, no probabilidad. Para priorizar con cabeza se combina con:

- **EPSS:** probabilidad estimada (0–100 %) de que una vulnerabilidad se explote en los próximos 30 días.
- **Catálogo KEV de CISA:** lista de vulnerabilidades que ya se están explotando. Si está en KEV, va la primera.
- **Avisos de INCIBE-CERT y CCN-CERT:** la fuente en español para enterarse a tiempo.

El valor orienta la prioridad de parcheo, pero el contexto de la organización importa tanto como la puntuación.

### 3. Ciclo de gestión de vulnerabilidades

La gestión de vulnerabilidades es un proceso continuo compuesto por varias fases:

#### 3.1. Descubrimiento

- Inventario de herramientas, sistemas y versiones.
- Escáneres de vulnerabilidades (Nessus, OpenVAS, Qualys).
- Revisión de avisos de seguridad de fabricantes.

#### 3.2. Análisis

Para cada vulnerabilidad detectada:

- ¿Afecta realmente al sistema?
- ¿Es explotable desde Internet?
- ¿Tiene exploit público?
- ¿Cuál es el impacto?

#### 3.3. Priorización

No todas las vulnerabilidades pueden parchearse al momento; se prioriza según:

- Criticidad CVSS
- Exposición (Internet, interna, aislada)
- Importancia del activo
- Impacto en operación
- Existencia de exploit activo

#### 3.4. Mitigación y parcheo

Opciones:

- Instalar parche disponible
- Aplicar workaround temporal
- Deshabilitar servicio afectado

- Aislar el sistema hasta el parcheo
- Actualizar a una versión mayor

### 3.5. Verificación

Tras parchear:

- Escaneo de validación
- Revisión de logs
- Confirmación de que no se ha roto funcionalidad

## 4. Gestión de parches en sistemas operativos

### 4.1. Linux

- Parches de seguridad del kernel y paquetes
- Repositorios oficiales y firmados
- Automatización (unattended-upgrades, Ansible, SaltStack)
- Reinicios planificados para actualizaciones críticas

### 4.2. Windows

- Windows Update, y para gestión centralizada Intune / Windows Autopatch o Azure Update Manager (WSUS sigue funcionando, pero Microsoft ya no lo desarrolla)
- Programación de mantenimiento
- Parches de seguridad mensuales (Patch Tuesday)
- Reinicios automáticos o controlados según políticas

### 4.3. Buenas prácticas

- Evitar versiones EOL (End of Life)
- Mantener repositorios actualizados
- Revisar software que carga drivers o servicios

## 5. Gestión de parches en software y aplicaciones

### 5.1. Aplicaciones corporativas

- Actualizaciones regulares
- Validación en entorno de pruebas
- Licencias y versiones estables
- Documentación de cambios

### 5.2. Navegadores y plugins

- Actualizaciones automáticas obligatorias

- Deshabilitar plugins obsoletos
- Revisar extensiones maliciosas

### 5.3. Librerías y dependencias

Especialmente en desarrollo:

- Gestión con herramientas como npm audit, pip-audit, Dependabot, Trivy o Snyk
- Inventario de componentes (**SBOM**) para saber en minutos si una librería vulnerable está en tus aplicaciones
- Actualización de dependencias vulnerables
- Verificación de integridad de paquetes

## 6. Parcheo de infraestructura y servicios

### 6.1. Bases de datos

- Parches críticos frecuentes
- Ventanas de mantenimiento controladas
- Actualizaciones obligatorias en versiones gestionadas (Amazon RDS, Azure SQL)

### 6.2. Dispositivos de red

Firewalls, switches, routers:

- Firmware firmado
- Actualizaciones programadas
- Deshabilitar servicios inseguros (Telnet, SNMPv1)

### 6.3. Sistemas cloud

- Parches automáticos en servicios gestionados
- Gestión de AMIs/Imágenes actualizadas
- Rotación periódica de instancias antiguas

## 7. Priorización realista del parcheo

La priorización debe responder a tres preguntas clave:

### 1. ¿Hasta qué punto está expuesto?

- Expuesto a Internet
- En entorno DMZ
- Solo accesible internamente
- Aislado en red privada

## 2. ¿De qué depende el negocio?

Sistemas críticos se parchean antes que sistemas secundarios.

## 3. ¿Existe exploit público o ataque activo?

Si el exploit está circulando, se convierte en parche crítico.

## Plazos orientativos de parcheo

Cada organización fija los suyos en su política; estos son valores habituales:

| Situación                                            | Plazo                     |
|------------------------------------------------------|---------------------------|
| Crítica, o en KEV, en un sistema expuesto a Internet | 24-72 horas               |
| Alta en sistemas internos                            | 7 días                    |
| Media                                                | 30 días                   |
| Baja                                                 | 90 días o siguiente ciclo |

Si no se puede cumplir el plazo, se aplica un control compensatorio (aislar, filtrar, deshabilitar la función) y se documenta la excepción.

## 8. Automatización y orquestación

### 8.1. Herramientas

- Ansible
- Puppet
- Chef
- Intune / Windows Autopatch, WSUS
- MDM para móviles
- Sistemas de CI/CD para aplicaciones

### 8.2. Objetivos

- Evitar errores humanos
- Garantizar uniformidad
- Desplegar parches masivamente
- Reducir tiempos de exposición

## 9. Documentación del proceso

Un proceso maduro debe incluir:

- Inventario actualizado de sistemas
- Registro de versiones actuales y objetivo
- Ventanas de mantenimiento

- Historial de parches aplicados
- Excepciones justificadas
- Controles compensatorios

Todo cambio debe quedar reflejado para auditoría.

## **10. Buenas prácticas generales**

- Automatizar siempre que sea posible
- Parchear primero los sistemas expuestos a Internet
- Revisar CVEs semanalmente
- Evitar versiones antiguas EOL
- Validar que el parche no rompe producción
- Documentar excepciones
- Repetir el ciclo cada mes (mínimo)

### **Para ampliar**

- NVD ([nvd.nist.gov](https://nvd.nist.gov)) y [cve.org](https://cve.org): bases de datos oficiales de CVE.
- FIRST: especificación de CVSS 4.0 y modelo EPSS.
- CISA: catálogo Known Exploited Vulnerabilities (KEV).
- INCIBE-CERT y CCN-CERT: avisos y alertas de seguridad en español.
- NIST SP 800-40 rev. 4: planificación de la gestión de parches.